Scalability Analysis of Parallel Hyperparameter Optimization for Machine Learning on Resource-Constrained Multicore Hardware

Siam Abdullah
Brac University
ORCID: 0009-0003-4847-2576

Sultana Khairum Arefa
Brac University
ORCID: 0009-0008-1139-7775

Abstract

Hyperparameter optimization (HPO) trains and checks many model settings. These jobs can run in parallel, but adding workers does not always give the same amount of speedup. This paper studies parallel HPO on an Intel Core i5-1135G7 processor with four physical cores and eight logical processors. The test workload is a Random Forest grid search with 12 settings and five-fold stratified cross-validation. Each search completes 60 model fits. The outer worker count is set to 1, 2, 4, or 8 with the Joblib Loky backend. The Random Forest and the numerical libraries use one thread inside each worker. For the main 25,000-sample workload, called W25, mean search time falls from 126.30 s with one worker to 43.15 s with eight workers. This is a 65.83% runtime reduction and a speedup of 2.93. Efficiency falls to 36.59%, and moving from four to eight workers reduces runtime by only another 13.29%. The 100,000-sample workload, called W100, takes about 4.36 times longer with one worker but reaches a lower eight-worker speedup of 2.67. Thus, the larger workload does not scale better in this test. The best settings and cross-validation scores stay the same across worker counts. Separate reruns also show the same main patterns. Eight workers give the shortest tested time, while four workers use the available parallel capacity more efficiently.

Index Terms

hyperparameter optimization, multicore scalability, process parallelism, speedup, parallel efficiency, Random Forest, resource-constrained computing

I. Introduction

Hyperparameter optimization can take a long time because every candidate model must be trained and checked. The cost grows when the search space, number of cross-validation folds, or training set becomes larger [1], [2]. Many candidate checks are independent, so they can run as separate parallel tasks. This study uses grid search because the full set of jobs is fixed before each run. Other search methods may use a trial budget better when model quality is the main goal [3].

Independent tasks do not guarantee linear speedup. Starting processes, scheduling jobs, moving data, using memory, and running the serial parts of the program all take time. These costs limit the gain from extra workers [4], [5]. The limit is important on a laptop CPU because several processes share the same cores, caches, memory system, and other hardware resources.

Machine-learning software can also create parallel work at more than one level. The HPO program may start worker processes while the estimator or a numerical library starts more threads inside each worker. If both levels are active, the machine may run far more tasks than planned. This is called oversubscription. Scikit-learn explains this issue, and Joblib can limit the inner thread pools used with its Loky process backend [6], [7]. The experiment therefore limits inner threads and changes only the outer HPO worker count.

This project studies HPO as a parallel computing workload. It does not propose a new HPO algorithm. The Random Forest grid, cross-validation method, score, and inner-thread limits stay fixed. Only the number of outer worker processes changes. The research questions are:

RQ1: How does parallel worker count affect total HPO execution time?
RQ2: How do speedup and parallel efficiency change as workers increase?
RQ3: At what concurrency do overhead and resource contention produce diminishing or negative returns?
RQ4: How does training workload size affect parallel HPO scalability?

The hypotheses are: more workers first reduce runtime (H1); speedup stays below ideal linear scaling because of overhead and shared hardware use (H2); efficiency falls after the system reaches a region of diminishing returns (H3); and a larger training workload scales better because each fit does more work compared with the process and scheduling costs (H4).

The project provides a controlled worker-count test on a four-core, eight-thread laptop. It reports runtime, speedup, efficiency, throughput, CPU use, and process-tree memory use. It also checks whether parallel execution changes the selected model settings. Separate reruns are kept as reproducibility evidence and are not mixed with the main timing results.

II. Related Work

A. Hyperparameter Optimization and Parallel Execution

HPO searches for model settings by training and checking many candidates. Feurer and Hutter [1] explain the main ideas behind HPO. Bischl et al. [2] review newer methods, evaluation rules, runtime improvements, and open problems. Grid search and random search are two common basic methods. Bergstra and Bengio [3] found that random search can use a limited trial budget better when only a few parameters have a strong effect. This project does not claim that grid search is the best HPO method. It uses grid search because every benchmark run then has the same candidates and the same number of fits.

Many HPO studies change both the search method and the way jobs run. Snoek et al. [8] used parallel trials in Bayesian optimization. Hyperband gives different budgets to candidates and stops weak candidates early [9]. BOHB joins this budget method with model-based selection [10]. ASHA removes waiting points from successive halving and supports large distributed runs [11]. These studies show that HPO can use parallel resources. The current project asks a narrower systems question: how does the same cross-validated workload scale as worker count changes on one small shared-memory computer?

B. Parallel-Performance Context

Amdahl's law explains why the serial part of a fixed workload limits speedup [4]. Karp and Flatt [5] show why runtime, speedup, and efficiency should be studied together. Gustafson's scaled-workload view explains why adding useful work can change the balance between computation and fixed overhead [12]. These ideas help explain the results, but this experiment does not prove which hardware limit caused the slowdown. It is also not a formal test of either law.

The machine-learning workload uses Random Forest, introduced by Breiman [13], through scikit-learn [14]. The Covertype dataset contains map-based features for predicting seven forest-cover classes [15], [16]. It is large enough for repeated CPU training. However, this project studies parallel execution, not the forest-cover prediction problem itself.

Tables I and II contain the full literature matrix prepared for the project. Earlier work already covers parallel HPO and resource-aware search. This project focuses on worker-count scaling for one fixed search on a small shared-memory computer.

TABLE I
Literature Matrix, Part I

Source	Problem and method	HPC relevance and main finding	Limitation and relation to this study
Feurer and Hutter (2019)	Systematic tuning of expensive ML hyperparameters; review of black-box, Bayesian, and multi-fidelity HPO.	Establishes HPO as a costly repeated-evaluation workload; modern HPO may require substantial computation.	Does not isolate multicore worker scalability; supplies the general HPO framework and motivation.
Bergstra and Bengio (2012)	Efficiency of random versus grid search; empirical and theoretical comparison.	Grid evaluations are simple and naturally parallel; random search can be more efficient when few dimensions matter.	Focuses on search quality rather than CPU scaling; supports a fixed grid as a deterministic benchmark.
Snoek et al. (2012)	Automatic tuning of expensive ML algorithms using Gaussian-process Bayesian optimization.	Includes multicore parallel experimentation; concurrent trials can accelerate Bayesian optimization.	The search policy adapts during execution; provides direct precedent for parallel HPO.
Li et al. (2018), Hyperband	Avoiding resources spent on weak configurations through bandit allocation and early stopping.	Reduces computation assigned to poor trials; adaptive budgets can substantially reduce HPO cost.	Candidates receive unequal work; contrasts with the equal-work fixed-grid experiment.
Falkner et al. (2018), BOHB	Combining efficient search and adaptive evaluation through Bayesian optimization with Hyperband.	Designed for scalable, resource-efficient HPO and strong anytime behavior.	Search and resource policy change together; illustrates algorithmic rather than execution-only scaling.

TABLE II
Literature Matrix, Part II

Source	Problem and method	HPC relevance and main finding	Limitation and relation to this study
Li et al. (2020), ASHA	Large-scale distributed tuning using asynchronous successive halving and early stopping.	Explicitly targets many-worker parallelism and reports near-linear scaling in distributed tests.	Targets large distributed systems; motivates a worker-scaling study on small hardware.
Bischl et al. (2023)	Foundations and practice of modern HPO; comprehensive review.	Covers runtime improvement and parallelization; execution strategy is a practical HPO concern.	Provides no controlled commodity-CPU benchmark; supports execution-level analysis.
Amdahl (1967)	Limits of fixed-work acceleration through a theoretical performance model.	Supplies foundational fixed-workload context; serial work limits attainable speedup.	Does not identify modern bottlenecks; interprets sublinear speedup without proving its cause.
Karp and Flatt (1990)	Measurement of parallel performance using timing, speedup, efficiency, and serial-fraction analysis.	Directly defines evaluation concepts and shows why several metrics are needed.	Not specific to ML; supports the chosen scalability metrics.
Blackard and Dean (1999)	Forest-cover prediction from cartographic variables using neural networks and discriminant analysis.	Supplies the application workload rather than HPC theory; neural networks performed better in the reported comparison.	Provides no scalability analysis; establishes Covertype's domain provenance.

III. Methodology

Figure 1 shows the full work process. It starts with the fixed data split and nested workloads, then covers calibration, repeated benchmarks, analysis, and separate reruns. The generated results stay linked to their raw input files.

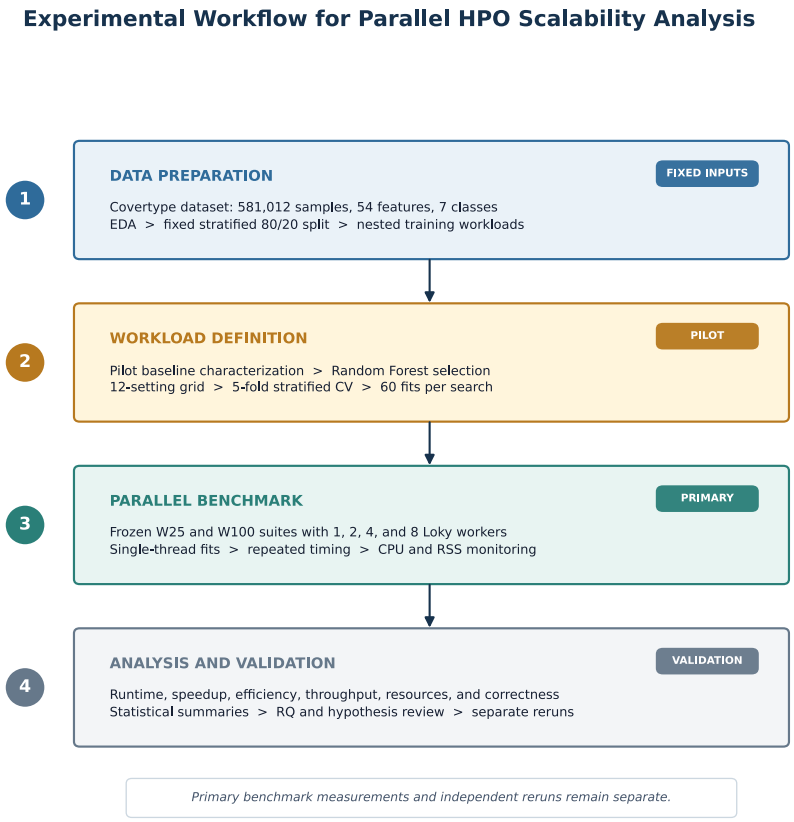


Fig. 1. Complete project workflow. Pilot characterization is separated from the frozen W25 and W100 benchmark suites, their analysis artifacts, and the independent reproduction runs.

A. Dataset, Split, and Training Workloads

The Covertype dataset has 581,012 rows, 54 input features, seven classes, and no missing values [15]. A fixed stratified 80/20 split with random_state=42 gives 464,809 training rows and 116,203 test rows. The final HPO searches and parameter selection use only the training part and cross-validation. Two nested training subsets are used in the measured tests. W25 has 25,000 rows, and W100 has 100,000 rows.

The class counts are uneven, so the main HPO score is mean Macro F1 across the validation folds. Early tests compared a Dummy Classifier, Logistic Regression, Random Forest, and Histogram Gradient Boosting. Random Forest was chosen because its training time was long enough to measure, its runs were repeatable, and it had a small useful parameter grid. This choice only defines the benchmark workload. It does not mean Random Forest is always the best model.

B. Exploratory Data Analysis

The 54 input features include 10 numeric map features and 44 binary wilderness-area or soil-type fields. No missing values or duplicate rows were found, so no filling or duplicate removal was needed. The data frame used about 243.8 MB in memory. Class 2 contains 48.76% of the rows and class 1 contains 36.46%. Classes 4 to 7 are much smaller. For this reason, Macro F1 is used instead of accuracy alone.

The numeric features were checked with distribution plots and a Pearson correlation matrix. The largest absolute correlation was between the 9 a.m. and 3 p.m. hillshade values ($r = -0.7803$). Aspect and 3 p.m. hillshade had $r = 0.6469$, while horizontal and vertical hydrology distance had $r = 0.6062$. These values describe the dataset only. No feature was removed only because it was correlated with another feature. Appendix A includes all seven EDA figures, with both normal and log-scale class plots.

TABLE III
Pilot Baseline Model Characterization

Model	Train (s)	Predict (s)	Accuracy	Macro F1	Weighted F1
Dummy Classifier	0.04	0.001	0.4876	0.0937	0.3197
Logistic Regression	26.48	0.057	0.7234	0.5308	0.7138
Random Forest	81.76	3.074	0.9533	0.9241	0.9531
Histogram Gradient Boosting	18.81	1.023	0.7877	0.7369	0.7858

Table III shows the early measurements used to choose the workload. Each value comes from one pilot run, not from the repeated final benchmark. An early Histogram Gradient Boosting test used automatic early stopping. It completed a different number of iterations for different workload sizes, so those times were not used as scaling results.

C. Fixed HPO Workload

The grid uses n_estimators values {50, 100, 200}, max_depth values {10, None}, and min_samples_leaf values {1, 4}. This gives 12 parameter settings. Five-fold StratifiedKFold, with shuffling and random_state=42, gives 60 fits in each complete search.

GridSearchCV uses refit=False during timing, so the search time does not include final model training. After the search, the chosen setting is fitted separately with a single-thread Random Forest. This also allows the selected result to be checked across worker counts.

D. Parallel Design and Thread Control

The tested worker counts are 1, 2, 4, and 8. GridSearchCV sends separate fits to Joblib's process-based Loky backend. Random Forest uses n_jobs=1. The variables OMP_NUM_THREADS, OPENBLAS_NUM_THREADS, MKL_NUM_THREADS, and NUMEXPR_NUM_THREADS are set to one. Joblib also uses inner_max_num_threads=1. These controls keep the outer worker count as the main changed variable. Figure 2 shows this setup.

E. Hardware and Software Environment

All measurements were collected on the same Windows 11 laptop. Its Intel Core i5-1135G7 has four physical cores and eight logical processors. Therefore, the eight-worker test goes beyond the physical-core count and uses the logical processors. The machine has 15.7 GB RAM. It also has an NVIDIA GeForce MX330, but the GPU was not used for HPO. Tables IV and V list the hardware and software versions.

F. Benchmark and Monitoring Protocol

W25 uses five repetitions for each worker count, giving 20 main runs. W100 uses three repetitions for each worker count, giving 12 main runs. Worker order is randomized inside each repetition block. Every run starts in a fresh Python process and is followed by a 10-second cooldown. Pilot, smoke, dry, and monitor-check runs are not included in the reported statistics.

A monitor checks the main benchmark process and all worker processes about every 250 ms. CPU use is divided by the eight logical processors. Memory is reported as the sum of resident set size (RSS) across the process tree. Shared

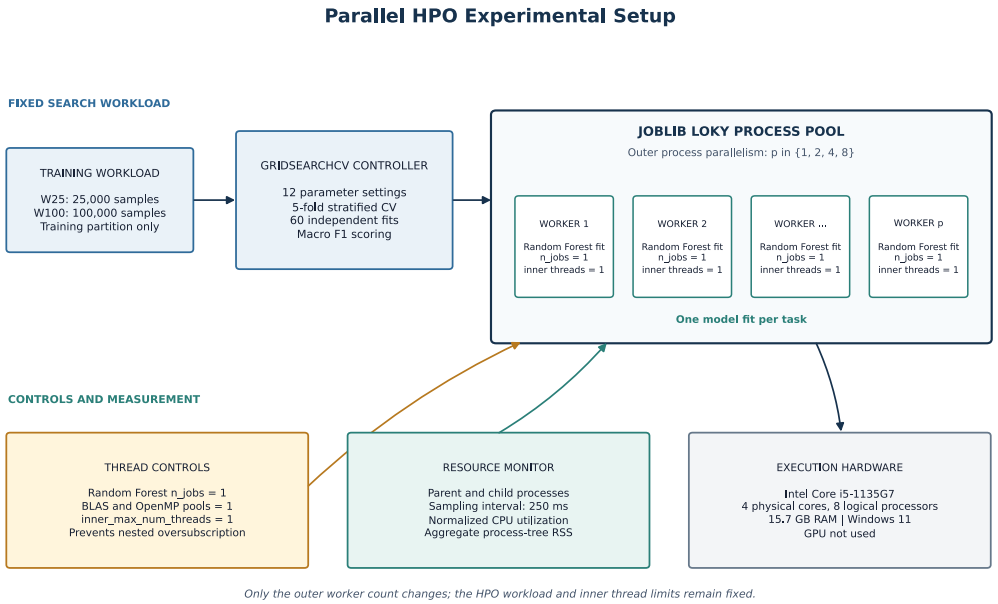


Fig. 2. Controlled parallel HPO setup. GridSearchCV sends independent fits to Loky processes. The estimator and numerical libraries use one inner thread. The monitor checks the main process and its workers.

TABLE IV
Experimental Hardware Environment

Component	Configuration
CPU	Intel Core i5-1135G7, 2.40 GHz
Physical CPU cores	4
Logical processors	8
System memory	15.7 GB RAM
GPU	NVIDIA GeForce MX330, 2,048 MiB
GPU usage	Not used for HPO
Operating system	Windows 11 Home Single Language, 64-bit
OS build	10.0.26200

TABLE V
Experimental Software Environment

Software	Version
Python	3.11.15
NumPy	2.4.6
pandas	3.0.5
SciPy	1.17.1
scikit-learn	1.9.0
Joblib	1.5.3
threadpoolctl	3.6.0
psutil	7.2.2

pages can be counted in more than one process. For this reason, the RSS value is only an approximate process-memory indicator, not an exact measure of unique physical RAM use.

Tables VI and VII give the full experimental settings and the fixed benchmark schedule. They separate the timed search from the manual refit and show the different numbers of runs for W25 and W100.

G. Metrics and Statistical Treatment

Let $\overline{T}_1$ be mean search time at one worker and $\overline{T}_p$ the mean at p workers. Speedup and parallel efficiency are

$$S_p = \frac{\overline{T}_1}{\overline{T}_p}, \qquad E_p = \frac{S_p}{p}. \tag{1}$$

TABLE VI
Complete Parallel HPO Experimental Configuration

Parameter	Configuration
Dataset	Forest Covertypes (fetch_covtype)
Total observations	581,012
Input features	54
Target classes	7
Fixed split	80% train / 20% test, stratified
Training partition	464,809 observations
Test partition	116,203 observations
Split seed	random_state=42
Primary workloads	W25 = 25,000; W100 = 100,000
Estimator	RandomForestClassifier
n_estimators	50, 100, 200
max_depth	10, None
min_samples_leaf	1, 4

Parameter	Configuration
HPO configurations	12
Cross-validation	5-fold StratifiedKFold
CV shuffle / seed	True / random_state=42
Scoring metric	Macro F1 (f1_macro)
CV fits per HPO run	60
Worker counts	1, 2, 4, 8
Parallel backend	Joblib Loky
Estimator parallelism	n_jobs=1
Inner numerical threads	1 per worker
GridSearchCV refit	False during timed search
Final refit	Separate single-thread manual refit
Resource sample interval	250 ms
Inter-run cooldown	10 s

TABLE VII
Frozen Primary Benchmark Protocol

Workload	Training samples	Workers	Repetitions/worker	Final runs	Randomization seed	Experiment label
W25	25,000	1, 2, 4, 8	5	20	4492026	final_v1
W100	100,000	1, 2, 4, 8	3	12	4496100	w100_v1

Equation 1 defines speedup and parallel efficiency. Speedup is calculated from the mean runtimes. It is not the mean of separate speedup values from each run. Throughput is 60 completed fits divided by search time. Runtime summaries include the mean, sample standard deviation, and a two-sided Student-t 95% confidence interval. Paired t-tests compare worker counts inside the repetition blocks. These tests are supporting evidence only because W25 has five pairs and W100 has three.

For the correctness check, each workload must return one best parameter set and no change in its best cross-validation Macro F1 score. The separate W25 and W100 reruns use the same methods, but their measurements are not added to the main benchmark samples.

IV. Results

A. Primary W25 Runtime and Speedup

Table VIII shows the main W25 results. Mean runtime falls at every worker level. It is 126.30 s with one worker, 74.97 s with two, 49.76 s with four, and 43.15 s with eight. Moving from one to eight workers saves 83.15 s, which is a 65.83% reduction. Figure 3 shows the mean times and their 95% confidence intervals.

TABLE VIII
Primary W25 Parallel HPO Results

Workers	Runtime (s)	Speedup	Efficiency	Fits/s
1	126.30	1.000	100.00%	0.475
2	74.97	1.685	84.23%	0.802
4	49.76	2.538	63.45%	1.213
8	43.15	2.927	36.59%	1.404

The speedup is useful but much lower than ideal linear scaling, as shown in Fig. 4. Eight workers would give a speedup of 8 under ideal scaling, but the measured value is 2.927. Throughput still rises from 0.475 to 1.404 fits/s. Extra workers continue to finish more fits per second, but the added gain becomes smaller.

B. Efficiency, Resource Demand, and Diminishing Returns

Parallel efficiency falls each time more workers are added, as shown in Fig. 5. It drops from 63.45% with four workers to 36.59% with eight. During the same step, runtime falls from 49.76 to 43.15 s, which is only a further 13.29% reduction. Eight workers are still the fastest tested setting. This means the result shows diminishing returns, not negative scaling.

Figure 6 shows that more workers still complete more fits each second. Table IX and Figs. 7 to 8 show the related resource use. For W25, mean normalized process-tree CPU use rises from about 12.5% with one worker to 79.1% with eight. Mean peak aggregate RSS rises from 596 to 2,585 MB. W100 follows the same pattern and reaches about

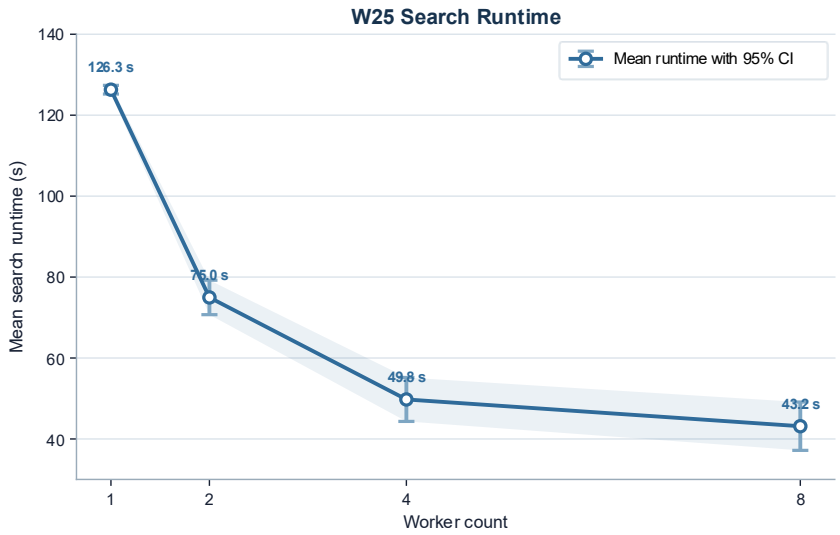


Fig. 3. Mean W25 search runtime with Student-t 95% confidence intervals over five repetitions per worker count.

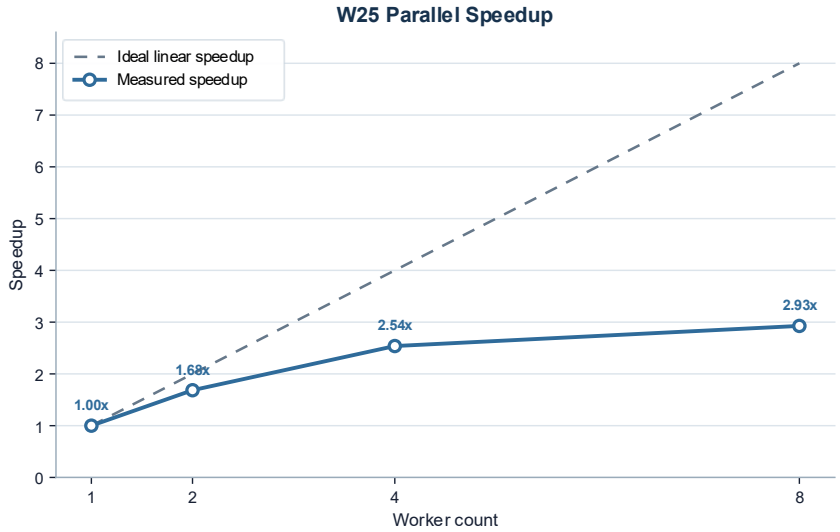


Fig. 4. Measured W25 speedup against the ideal linear reference. The gap widens as workers increase.

4,768 MB with eight workers. These results show higher CPU and memory demand. They do not show whether cache use, memory bandwidth, or another hardware limit caused the sublinear speedup.

TABLE IX
Measured Process-Tree Resource Indicators

	CPU (%)		Peak RSS (MB)	
Workers	W25	W100	W25	W100
1	12.52	12.54	596	1,027
2	24.56	24.61	1,058	1,780
4	46.88	47.37	1,741	3,261
8	79.14	80.02	2,585	4,768

C. Training-Workload Size

The one-worker W100 search takes 550.15 s, about 4.36 times the W25 time. H4 expected the extra work in each W100 fit to make process and scheduling costs less important. If that happened, W100 would have shown better

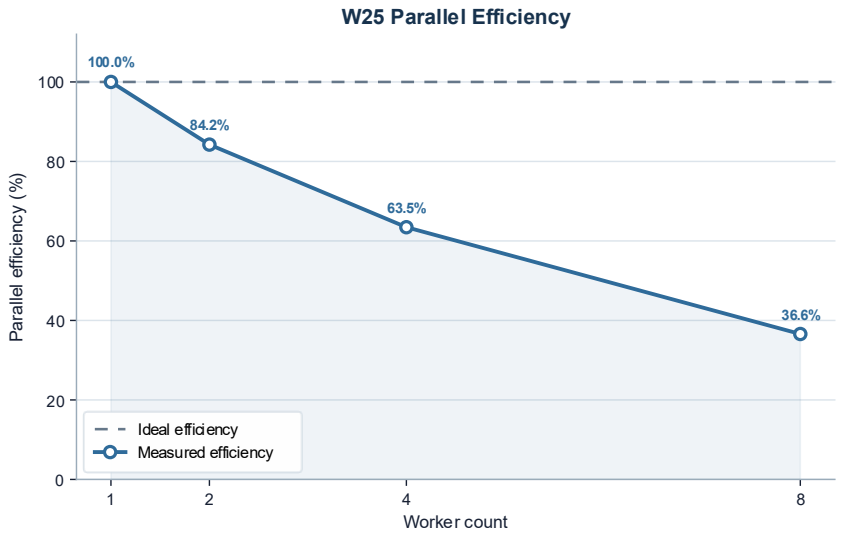


Fig. 5. W25 parallel efficiency relative to ideal 100% efficiency.

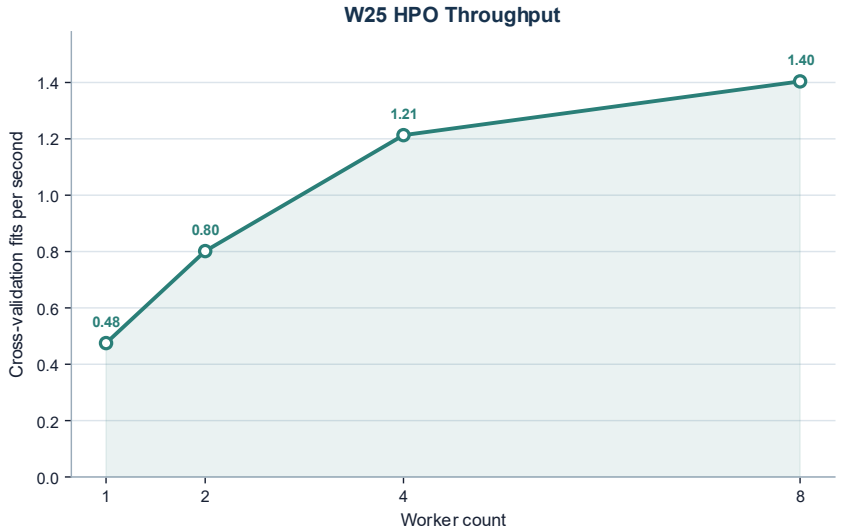


Fig. 6. W25 throughput in completed cross-validation fits per second. Throughput continues to rise even where efficiency falls sharply.

speedup. Table X and Fig. 9 show that W100 instead has slightly lower speedup at every parallel worker count. Table XI gives the complete W100 runtime summary.

TABLE X
W25 and W100 Scaling Comparison

	Speedup		Efficiency (%)	
Workers	W25	W100	W25	W100
1	1.000	1.000	100.00	100.00
2	1.685	1.647	84.23	82.37
4	2.538	2.342	63.45	58.54
8	2.927	2.667	36.59	33.34

With eight workers, W100 reaches a speedup of 2.667 and efficiency of 33.34%. W25 reaches 2.927 speedup and 36.59% efficiency. Figure 10 shows the same order with two and four workers. The larger workload does not give the expected scaling benefit. The W25 and W100 tests were run in separate sessions, and only two workload sizes were tested. The comparison is therefore descriptive and does not mean that larger HPO workloads always scale worse.

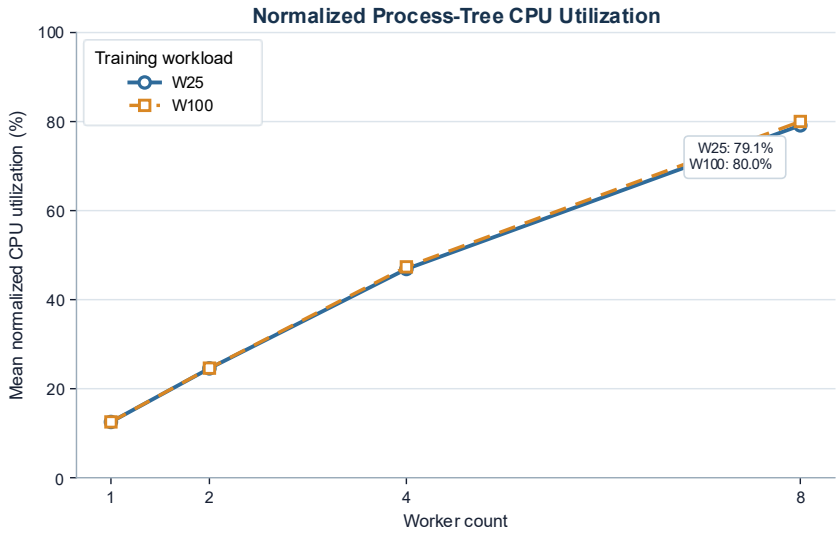


Fig. 7. Mean normalized process-tree CPU utilization for W25 and W100. Equivalent worker counts produce similar normalized CPU levels.

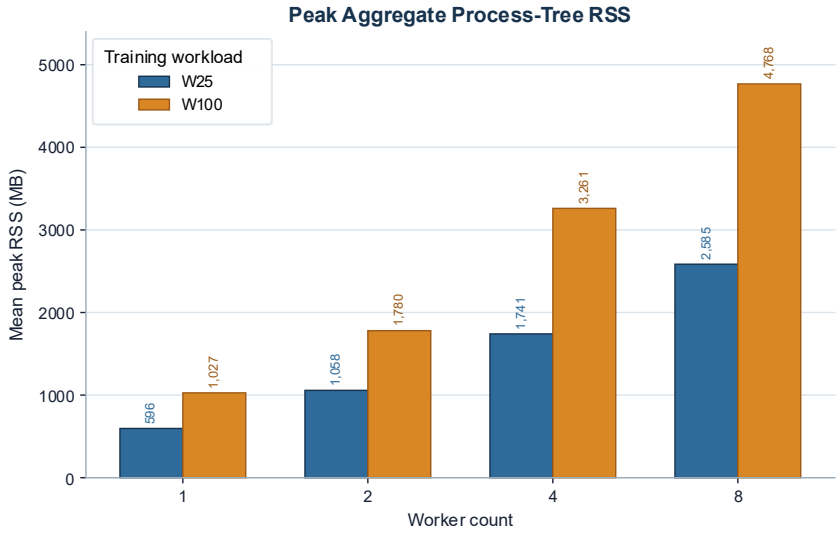


Fig. 8. Mean peak aggregate process-tree RSS for W25 and W100. RSS is an approximate process-memory indicator and can double-count shared pages.

TABLE XI
Primary W100 Parallel HPO Results

Workers	Runtime (s)	95% CI (s)	Speedup	Efficiency	Fits/s
1	550.15	[548.42, 551.88]	1.000	100.00%	0.109
2	333.96	[325.17, 342.75]	1.647	82.37%	0.180
4	234.95	[227.35, 242.55]	2.342	58.54%	0.255
8	206.26	[189.13, 223.39]	2.667	33.34%	0.291

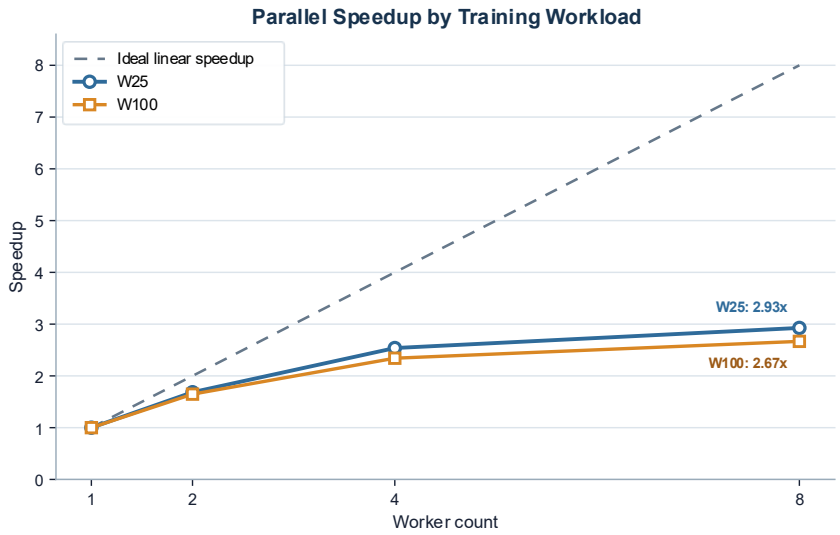


Fig. 9. Speedup by training-workload size. W100 performs more absolute work but scales slightly worse than W25 at 2, 4, and 8 workers.

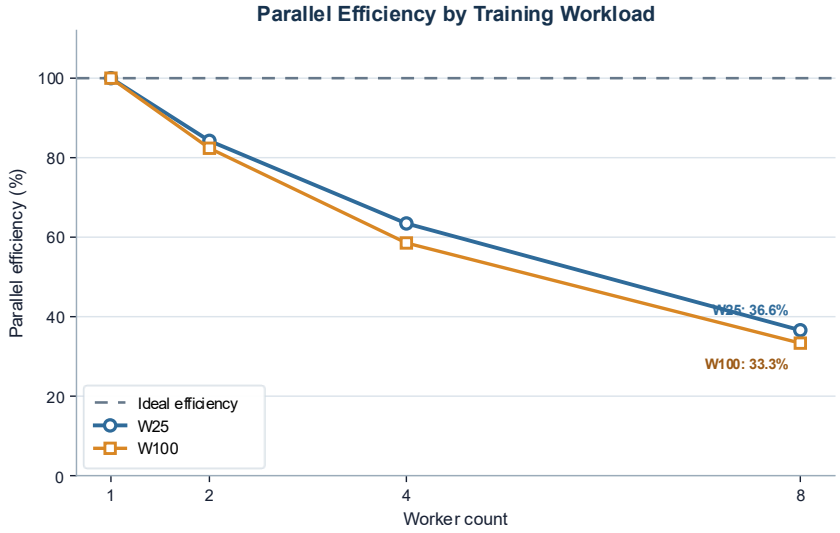


Fig. 10. Parallel efficiency by training-workload size. W100 remains slightly below W25 at each parallel worker count.

D. Correctness, Statistical Evidence, and Reruns

All 20 main W25 runs select max_depth=None, min_samples_leaf=1, and n_estimators=100. Their best cross-validation Macro F1 is 0.746271. All 12 main W100 runs select the same depth and leaf size with n_estimators=200. Their best score is 0.843836. The worker count changes runtime but does not change the selected result inside either workload. Table XII summarizes these results.

TABLE XII
HPO Correctness Summary Across Primary Runs

Workload	Runs	Unique parameter sets	max_depth	min_samples_leaf	n_estimators	Best Macro F1	Score range
W25	20	1	None	1	100	0.746271	0
W100	12	1	None	1	200	0.843836	0

Table XIII gives the supplementary paired comparisons. The W25 comparison from four to eight workers gives a mean reduction of 6.61 s and a two-sided p-value of 0.039. For W100, the same comparison reduces mean time by

28.69 s with $p = 0.024$. These tests use only five and three pairs. They are included as supporting evidence and are not used for broad claims about cause.

TABLE XIII
Supplementary Paired Runtime Comparisons

Workload	From	To	Pairs	Mean reduction (s)	Reduction (%)	95% CI of difference (s)	Paired t	p-value / Cohen's d_z
W25	1	2	5	51.331	40.642	[48.132, 54.530]	44.546	1.52×10^{-6} / 19.922
W25	2	4	5	25.207	33.623	[17.730, 32.684]	9.360	0.000726 / 4.186
W25	4	8	5	6.611	13.285	[0.518, 12.704]	3.013	0.03945 / 1.347
W25	1	8	5	83.149	65.835	[76.892, 89.406]	36.895	3.22×10^{-6} / 16.500
W100	1	2	3	216.191	39.297	[205.677, 226.705]	88.472	0.000128 / 51.080
W100	2	4	3	99.007	29.646	[83.628, 114.386]	27.699	0.00130 / 15.992
W100	4	8	3	28.688	12.210	[9.251, 48.126]	6.351	0.02391 / 3.666
W100	1	8	3	343.887	62.508	[327.584, 360.189]	90.759	0.000121 / 52.400

The separate reruns again show lower runtime with more workers, sublinear speedup, falling efficiency, the same selected parameters, and the same cross-validation scores. Depending on worker count, W25 rerun means differ from the main means by about 0.7% to 5.9%. W100 differences are about 0.3% to 1.5%. The reruns also keep W25 speedup above W100 speedup at every parallel worker count. They are used only to check reproducibility and are not added to the main sample sizes.

V. Discussion

The results answer the four research questions. For RQ1, more workers reduce HPO time in every tested W25 and W100 setting. For RQ2, speedup rises but stays below linear scaling, while efficiency falls. For RQ3, the clearest diminishing returns appear between four and eight workers. Eight workers are still faster, so negative scaling does not occur. For RQ4, the larger W100 workload does not scale better.

TABLE XIV
Hypothesis Outcomes

Hyp.	Outcome
H1	Supported: added workers reduce runtime.
H2	Supported: measured speedup is sublinear.
H3	Supported with qualification: efficiency declines and diminishing returns appear, but negative scaling is not observed.
H4	Not supported under the tested conditions: W100 scales slightly worse than W25.

Table XIV summarizes the hypothesis results. There is no single worker count that is best for every goal. Eight workers give the shortest search time in this experiment. Four workers give a speedup of 2.54 with 63.45% efficiency, so they provide a better balance when efficient resource use matters. These points apply only to the tested machine, grid, model, and workloads.

CPU use and RSS both rise with worker count. This agrees with the idea that more processes must share limited resources, but the measurements cannot identify one exact cause for the efficiency loss. Cache, memory bandwidth, context-switch, and power measurements would be needed for that work.

The workload comparison shows that a longer task does not automatically scale better. W100 does more than four times the one-worker work but does not improve speedup. More workload sizes and lower-level measurements are needed to separate model fitting, data access, scheduling, and shared-resource costs.

VI. Validity and Reproducibility

The study has clear limits. It uses one laptop, one operating system, one model family, one dataset, one grid, and four worker counts. The results form a reproducible case study, not a rule for every computer. W25 has five runs for each worker count, while W100 has three. The confidence intervals and paired tests, especially for W100, must be read with these small sample sizes in mind. W25 and W100 were also measured in separate sessions, so their comparison is descriptive and not a paired statistical test.

The test partition was used during early pilot baseline checks. It was not used in the final HPO search or parameter selection, which used the training partition and cross-validation only. Therefore, the paper does not claim that the test set stayed completely untouched. The main conclusions are about computing performance, not a final untouched estimate of prediction quality.

The resource measurements also have limits. Normalized process-tree CPU use comes from software samples. Aggregate process-tree RSS may count shared pages more than once. Neither value is an exact measure of physical resource ownership. The GPU was not used. The processor has four physical cores and eight logical processors.

Reproducibility steps include fixed split and workload index files, fixed random seeds, inner-thread limits, randomized worker order, fresh benchmark processes, saved raw outputs, analysis scripts, package versions, and a SHA-256 file list. The experimental source was frozen at Git commit b30c645. A later provenance record points back to that frozen state. The main and reproduction folders stay separate, so the reruns do not increase the main sample size.

VII. Conclusion

Process-level parallelism clearly reduces the time needed for the fixed Random Forest HPO workload on the tested laptop. For W25, eight workers reduce mean search time from 126.30 to 43.15 s. This gives a speedup of 2.93 and a runtime reduction of 65.83%. The speedup is sublinear. Efficiency falls to 36.59%, and moving from four to eight workers gives only another 13.29% runtime reduction. Negative scaling is not observed.

The larger W100 workload takes about 4.36 times longer with one worker but scales slightly worse. H4 is therefore not supported under the tested conditions. The selected HPO result stays correct across worker counts. Separate reruns support the same main findings and are not mixed with the main results.

For this hardware and workload, eight workers are useful when the shortest time is the main goal. Four workers give a better efficiency balance. Future work can test more processors, workload sizes, models, and HPO methods. It can also compare processes with threads and measure cache use, memory bandwidth, context switches, and energy.

Appendix A
Complete Exploratory-Analysis Figure Set

This appendix includes every EDA figure made for the project. Figures 11 and 12 show the same class counts on normal and log scales. Figures 13 to 16 show four numeric feature distributions. Figure 17 gives the correlation matrix for the ten non-binary features. These plots describe the dataset. They are not evidence of a hardware cause for the scaling results.

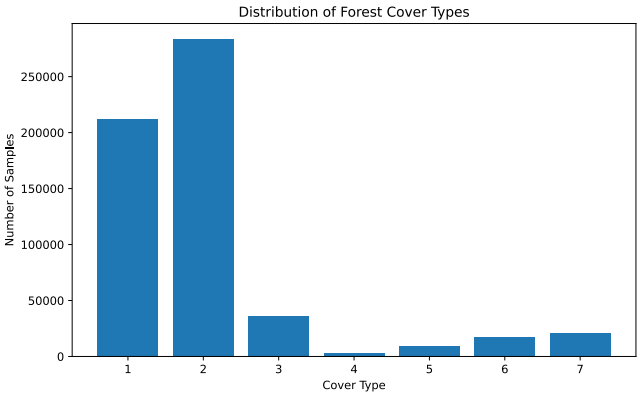


Fig. 11. Covertype class counts on a linear scale. Classes 1 and 2 are the largest.

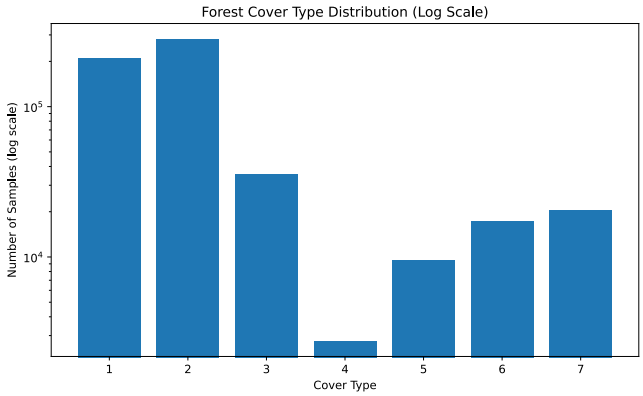


Fig. 12. The same class counts on a log scale, which makes the smaller classes easier to see.

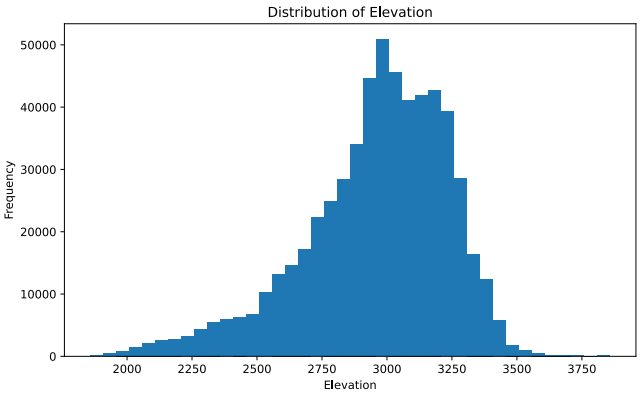


Fig. 13. Elevation values in the full Covertype dataset.

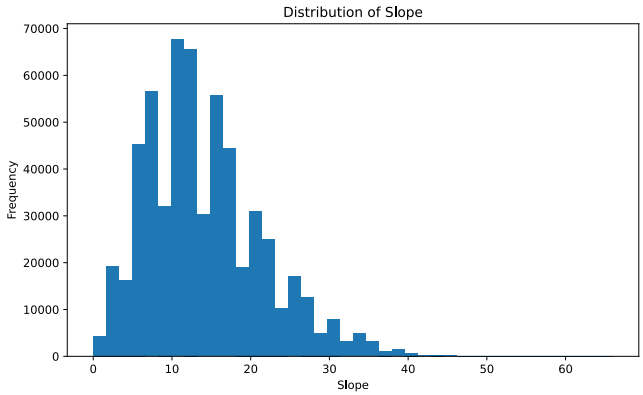


Fig. 14. Slope values in the full Covertype dataset.

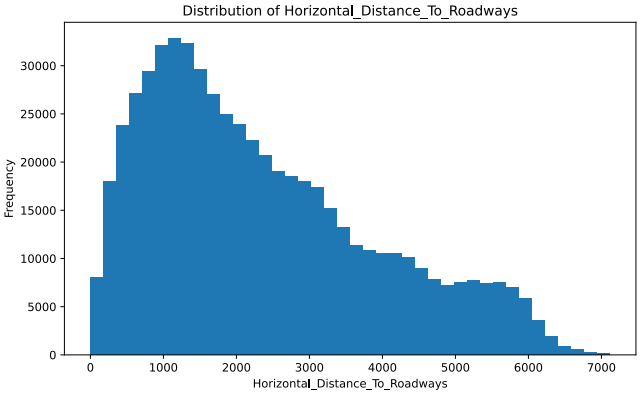


Fig. 15. Horizontal distance to roadways.

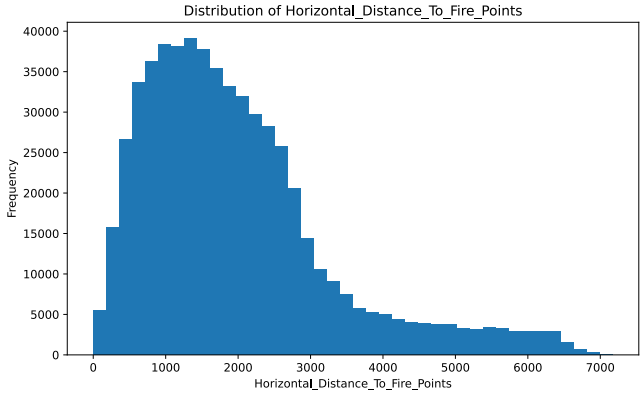


Fig. 16. Horizontal distance to fire points.

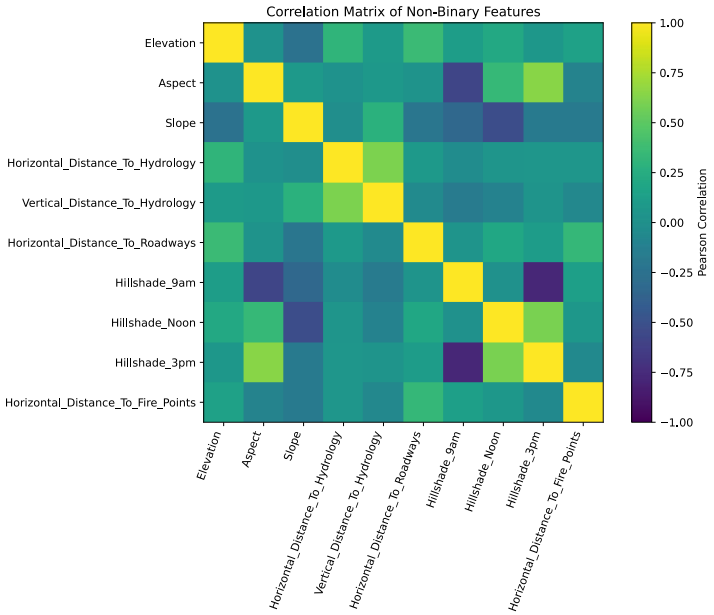


Fig. 17. Pearson-correlation matrix for the ten non-binary cartographic variables. Correlations are retained as descriptive EDA and were not used for automatic feature removal.

References

[1] M. Feurer and F. Hutter, "Hyperparameter optimization," in Automated Machine Learning: Methods, Systems, Challenges, F. Hutter, L. Kotthoff, and J. Vanschoren, Eds. Springer, 2019, p. 3-33.

[2] B. Bischl et al., "Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges," WIREs Data Mining and Knowledge Discovery, vol. 13, no. 2, p. e1484, 2023.

[3] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," Journal of Machine Learning Research, vol. 13, no. 10, p. 281-305, 2012.

[4] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in Proceedings of the April 18-20, 1967, Spring Joint Computer Conference. Association for Computing Machinery, 1967, p. 483-485.

[5] A. H. Karp and H. P. Flatt, "Measuring parallel processor performance," Communications of the ACM, vol. 33, no. 5, p. 539-543, 1990.

[6] scikit-learn developers, Parallelism, Resource Management, and Configuration, scikit-learn Documentation, version 1.9.0, accessed Aug. 30, 2026. [Online]. Available: https://scikit-learn.org/stable/computing/parallelism.html

[7] Joblib, Embarrassingly Parallel for Loops, Joblib Documentation, version 1.5.3, accessed Aug. 30, 2026. [Online]. Available: https://joblib.readthedocs.io/en/stable/parallel.html

[8] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in Advances in Neural Information Processing Systems, vol. 25, 2012, p. 2951-2959.

[9] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," Journal of Machine Learning Research, vol. 18, no. 185, p. 1-52, 2018.

[10] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and efficient hyperparameter optimization at scale," in Proceedings of the 35th International Conference on Machine Learning, vol. 80. Proceedings of Machine Learning Research, 2018, p. 1437-1446.

[11] L. Li et al., "A system for massively parallel hyperparameter tuning," Proceedings of Machine Learning and Systems, vol. 2, p. 230-246, 2020.

[12] J. L. Gustafson, "Reevaluating amdahl's law," Communications of the ACM, vol. 31, no. 5, p. 532-533, 1988.

[13] L. Breiman, "Random forests," Machine Learning, vol. 45, p. 5-32, 2001.

[14] F. Pedregosa et al., "Scikit-learn: Machine learning in python," Journal of Machine Learning Research, vol. 12, no. 85, p. 2825-2830, 2011.

[15] J. Blackard, "Covertype," UCI Machine Learning Repository, 1998.

[16] J. A. Blackard and D. J. Dean, "Comparative accuracies of artificial neural networks and discriminant analysis in predicting forest cover types from cartographic variables," Computers and Electronics in Agriculture, vol. 24, no. 3, p. 131-151, 1999.